

Register Number: 192425317

Name: M. Sahasra

CO2 – AT3

MLA0201 – Fundamentals of Machine Learning

Debugging & Optimisation Task

Q1. A real estate company developed a linear regression model to predict house prices, but the model is showing high prediction error.

- (i) Identify possible causes for the high error (e.g., multicollinearity, outliers, poor feature selection).
- (ii) Analyze the dataset/model behavior to detect these issues.
- (iii) Apply suitable optimization techniques (such as feature scaling, removing outliers, or adding relevant features) to improve the model performance.

Answer)

Definitions:

Linear Regression:

Linear Regression is a supervised learning algorithm used to predict a continuous numerical value based on one or more input features.

Multicollinearity:

Multicollinearity occurs when two or more independent features are highly correlated with each other. It can make regression coefficients unstable.

Outlier:

An outlier is an unusually large or small observation that is significantly different from the other data points.

Feature Selection:

Feature selection means choosing the most useful input variables for building a machine learning model.

Mean Squared Error (MSE):

MSE measures the average squared difference between actual and predicted values.

(i) Possible causes

The high prediction error can be caused by:

- Highly correlated features.
- Outliers in the house-price data.
- Irrelevant features.
- Missing important features.
- Different feature scales.
- Insufficient training data.
- Incorrect model assumptions.

(ii) Analysis

We can check:

- Correlation between features.
- MSE and R^2 score.
- Outliers using the IQR method.
- Training and testing errors.

(iii) Optimization

We can improve the model by:

- Removing extreme outliers.
- Removing highly correlated features.
- Scaling numerical features.
- Adding relevant features.
- Retraining the model.

Python Code:

```
1 # Q1: Debugging Linear Regression with High Prediction Error
2
3 import numpy as np
4 import pandas as pd
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.linear_model import LinearRegression
8 from sklearn.preprocessing import StandardScaler
9 from sklearn.metrics import mean_squared_error, r2_score
10
11 # -----
12 # 1. Create sample dataset
13 # -----
14 np.random.seed(42)
15
16 data = pd.DataFrame({
17     "Area": np.random.randint(800, 3000, 100),
18     "Bedrooms": np.random.randint(1, 6, 100),
19     "Age": np.random.randint(1, 30, 100)
20 })
21
22 # Create a correlated feature
23 data["Area_Copy"] = data["Area"] * 0.98 + np.random.normal(0, 20, 100)
24
25 # House price
26 data["Price"] = (
27     data["Area"] * 300
28     + data["Bedrooms"] * 50000
29     - data["Age"] * 5000
30     + np.random.normal(0, 50000, 100)
31 )
32
33 # Add an outlier
34 data.loc[0, "Price"] = 3000000
35
36 print("Sample Input Dataset:")
37 print(data.head())
38
39 # -----
40 # 2. Check correlation
41 # -----
42 print("\nCorrelation Matrix:")
43 print(data.corr(numeric_only=True))
44
45 # -----
46 # 3. Train initial model
47 # -----
48 X = data[["Area", "Bedrooms", "Age", "Area_Copy"]]
49 y = data["Price"]
50
51 X_train, X_test, y_train, y_test = train_test_split(
52     X, y, test_size=0.2, random_state=42
53 )
54
55 model = LinearRegression()
56 model.fit(X_train, y_train)
57
58 predictions = model.predict(X_test)
59
```

```

60 print("\nBefore Optimization:")
61 print("MSE:", round(mean_squared_error(y_test, predictions), 2))
62 print("R2 Score:", round(r2_score(y_test, predictions), 4))
63
64 # -----
65 # 4. Remove outlier using IQR
66 # -----
67 Q1 = data["Price"].quantile(0.25)
68 Q3 = data["Price"].quantile(0.75)
69
70 IQR = Q3 - Q1
71
72 lower = Q1 - 1.5 * IQR
73 upper = Q3 + 1.5 * IQR
74
75 clean_data = data[
76     (data["Price"] >= lower) &
77     (data["Price"] <= upper)
78 ].copy()
79
80 # -----
81 # 5. Remove correlated feature
82 # -----
83 X_clean = clean_data[["Area", "Bedrooms", "Age"]]
84 y_clean = clean_data["Price"]
85
86 X_train, X_test, y_train, y_test = train_test_split(
87     X_clean, y_clean, test_size=0.2, random_state=42
88 )
89
90 # -----
91 # 6. Feature scaling
92 # -----
93 scaler = StandardScaler()
94
95 X_train_scaled = scaler.fit_transform(X_train)
96 X_test_scaled = scaler.transform(X_test)
97
98 optimized_model = LinearRegression()
99 optimized_model.fit(X_train_scaled, y_train)
100 |
101 optimized_predictions = optimized_model.predict(X_test_scaled)
102
103 print("\nAfter Optimization:")
104 print("MSE:", round(
105     mean_squared_error(y_test, optimized_predictions), 2
106 ))
107 print("R2 Score:", round(
108     r2_score(y_test, optimized_predictions), 4
109 ))

```

Output:

... Sample Input Dataset:

	Area	Bedrooms	Age	Area_Copy	Price
0	1660	2	1	1621.506863	3.000000e+06
1	2094	2	8	2106.523383	6.689343e+05
2	1930	4	7	1903.913347	7.496759e+05
3	1895	5	18	1839.956849	7.616065e+05
4	2438	3	8	2367.822150	9.207008e+05

Correlation Matrix:

	Area	Bedrooms	Age	Area_Copy	Price
Area	1.000000	0.126632	-0.122233	0.999369	0.591684
Bedrooms	0.126632	1.000000	0.117884	0.120724	0.225287
Age	-0.122233	0.117884	1.000000	-0.126811	-0.281552
Area_Copy	0.999369	0.120724	-0.126811	1.000000	0.589721
Price	0.591684	0.225287	-0.281552	0.589721	1.000000

Before Optimization:

MSE: 292775635136.63

R2 Score: -0.101

After Optimization:

MSE: 1910799960.03

R2 Score: 0.9643

Result: Removing the outlier, reducing multicollinearity and scaling the features can improve the regression model.

Q2. An email classification system using a linear model is incorrectly classifying several spam emails as non-spam.

- Identify the reasons for misclassification (e.g., improper decision boundary, imbalanced data).
- Examine the current decision boundary and classification errors.
- Refine the model using optimization techniques such as adjusting thresholds, feature engineering, or re-training with balanced data.

Answer)

Definitions:**Classification:**

Classification is a supervised learning task where the model assigns data to predefined classes.

Spam Classification:

Spam classification identifies whether an email is spam or not spam.

Decision Boundary:

A decision boundary separates different classes in the feature space.

Class Imbalance:

Class imbalance occurs when one class has considerably more observations than another.

Classification Threshold:

A threshold converts predicted probabilities into class labels. In binary classification, 0.5 is commonly used as the default threshold.

(i) Causes

Possible causes include:

- Improper decision boundary.
- Imbalanced spam/non-spam data.
- Poor features.
- Default threshold being too high.
- Insufficient training data.

(ii) Analysis

We can examine:

- Confusion matrix.
- Precision.
- Recall.
- F1-score.
- Predicted probabilities.

For spam detection, recall is especially important because we want to detect as many spam emails as possible.

(iii) Optimization

We can:

- Lower the classification threshold.
- Add useful features.
- Balance the training data.
- Retrain the model.

Python Code:

```
1 # Q2: Spam Email Classification Optimization
2
3 import numpy as np
4 import pandas as pd
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.metrics import (
9     confusion_matrix,
10    classification_report
11 )
12 from sklearn.preprocessing import StandardScaler
13
14 # -----
15 # 1. Create sample dataset
16 # -----
17 np.random.seed(42)
18
19 data = pd.DataFrame({
20     "spam_words": np.random.randint(0, 15, 150),
21     "links": np.random.randint(0, 10, 150),
22     "capital_letters": np.random.randint(0, 20, 150)
23 })
24
25 # Generate labels
26 data["spam"] = (
27     (data["spam_words"] >= 7) |
28     (data["links"] >= 6)
29 ).astype(int)
30
31 print("Sample Input:")
32 print(data.head())
33
34 # -----
35 # 2. Split data
36 # -----
37 X = data[[
38     "spam_words",
39     "links",
40     "capital_letters"
41 ]]
--
```

```

43 y = data["spam"]
44
45 X_train, X_test, y_train, y_test = train_test_split(
46     X, y,
47     test_size=0.25,
48     random_state=42,
49     stratify=y
50 )
51
52 # -----
53 # 3. Train model
54 # -----
55 model = LogisticRegression()
56 model.fit(X_train, y_train)
57
58 probabilities = model.predict_proba(X_test)[:, 1]
59
60 # Default threshold
61 pred_default = (probabilities >= 0.5).astype(int)
62
63 print("\nBefore Optimization:")
64 print(confusion_matrix(y_test, pred_default))
65 print(classification_report(y_test, pred_default))
66
67 # -----
68 # 4. Optimize threshold
69 # -----
70 new_threshold = 0.35
71
72 pred_optimized = (
73     probabilities >= new_threshold
74 ).astype(int)
75
76 print("\nAfter Threshold Optimization:")
77 print(confusion_matrix(y_test, pred_optimized))
78 print(classification_report(y_test, pred_optimized))
79
80 # -----
81 # 5. Predict new email
82 # -----
83 new_email = [[10, 7, 15]]
84
85 probability = model.predict_proba(new_email)[0][1]
86
87 print("\nNew Email:")
88 print("Spam Probability:", round(probability, 3))
89
90 if probability >= new_threshold:
91     print("Prediction: SPAM")
92 else:
93     print("Prediction: NOT SPAM")

```


Output:

```
... Sample Input:
      spam_words  links  capital_letters  spam
0           6      6           0      1
1           3      9          19      1
2          12      8          10      1
3          14      6          16      1
4          10      8           7      1

Before Optimization:
[[ 8  1]
 [ 2 27]]
      precision    recall  f1-score   support

         0       0.80      0.89      0.84         9
         1       0.96      0.93      0.95        29

   accuracy                0.92        38
  macro avg       0.88      0.91      0.89        38
 weighted avg     0.93      0.92      0.92        38


After Threshold Optimization:
[[ 5  4]
 [ 1 28]]
      precision    recall  f1-score   support

         0       0.83      0.56      0.67         9
         1       0.88      0.97      0.92        29

   accuracy                0.87        38
  macro avg       0.85      0.76      0.79        38
 weighted avg     0.87      0.87      0.86        38


New Email:
Spam Probability: 1.0
Prediction: SPAM
```

Result: Lowering the threshold can increase the detection of spam emails.

Q3.A Naïve Bayes classifier is giving poor accuracy due to missing or zero probability values in the dataset.

- (i) Identify the issue related to missing or zero probabilities.
- (ii) Analyze how these values affect classification results.
- (iii) Apply appropriate debugging and optimization methods such as Laplace smoothing to improve accuracy.

Answer)

Definitions:**Naïve Bayes:**

Naïve Bayes is a probabilistic classification algorithm based on Bayes' theorem.

Conditional Probability:

Conditional probability represents the probability of an event occurring given another event.

Zero Probability Problem:

If a feature value never occurs with a particular class during training, its probability becomes zero.

Because Naïve Bayes multiplies probabilities, one zero probability can make the complete probability zero.

Laplace Smoothing:

Laplace smoothing adds a small value, usually 1, to frequency counts so that no probability becomes zero.

(i) Problem

Suppose:

$$P(\text{word} = \text{"offer"} \mid \text{spam}) = 0$$

Then the complete probability may become zero.

(ii) Effect

A zero probability can cause:

- Incorrect class probabilities.
- Poor classification.
- Complete elimination of a class.

(iii) Solution

Use Laplace smoothing.

Python Code:

```
1 # Q3: Naive Bayes and Laplace Smoothing
2
3 import pandas as pd
4 from sklearn.model_selection import train_test_split
5 from sklearn.naive_bayes import MultinomialNB
6 from sklearn.metrics import accuracy_score
7
8 # -----
9 # 1. Sample input
10 # -----
11 data = pd.DataFrame({
12     "offer": [5, 4, 0, 0, 3, 0, 4, 0],
13     "free": [4, 3, 0, 1, 5, 0, 4, 0],
14     "money": [3, 2, 0, 0, 4, 0, 3, 0],
15     "spam": [1, 1, 0, 0, 1, 0, 1, 0]
16 })
17
18 print("Input Dataset:")
19 print(data)
20
21 X = data[["offer", "free", "money"]]
22 y = data["spam"]
23
24 # -----
25 # 2. Train Naive Bayes
26 # alpha = 1 performs Laplace smoothing
27 # -----
28 model = MultinomialNB(alpha=1.0)
29
30 model.fit(X, y)
31
32 # -----
33 # 3. Test sample
34 # -----
35 new_email = [[2, 3, 2]]
36
37 prediction = model.predict(new_email)
38
39 print("\nNew Email Features:")
40 print(new_email)
41
42 print("Prediction:",
43       "SPAM" if prediction[0] == 1 else "NOT SPAM")
44
45 # -----
46 # 4. Show probabilities
47 # -----
48 probabilities = model.predict_proba(new_email)
49
50 print("\nClass Probabilities:")
51 print(probabilities)
52
53 print("\nAccuracy on Dataset:",
54       round(model.score(X, y), 3))
```

Output:

```
... Input Dataset:
      offer  free  money  spam
0         5    4      3     1
1         4    3      2     1
2         0    0      0     0
3         0    1      0     0
4         3    5      4     1
5         0    0      0     0
6         4    4      3     1
7         0    0      0     0

New Email Features:
[[2, 3, 2]]
Prediction: NOT SPAM

Class Probabilities:
[[0.50761161 0.49238839]]

Accuracy on Dataset: 0.875
```

Result: Laplace smoothing prevents zero probabilities and improves the stability of Naïve Bayes classification.

Q4. A discriminant function is unable to clearly separate two classes, leading to incorrect classifications.

- (i) Identify possible reasons for poor class separation (e.g., overlapping features, incorrect weights).
- (ii) Analyze the decision boundary and feature contribution.
- (iii) Improve the discriminant function using optimization strategies such as adjusting weights, feature transformation, or normalization.

Answer)

Definitions:

Discriminant Function:

A discriminant function calculates a score used to determine which class a data point belongs to.

A simple linear discriminant can be represented as:

Weight:

A weight represents the importance assigned to a feature.

Decision Boundary:

The decision boundary is the point where the discriminant score changes from one class to another.

Normalization:

Normalization converts features into a common range, often 0 to 1.

(i) Causes

- Incorrect feature weights.
- Features having different scales.
- Strong feature overlap.
- Irrelevant features.
- Poorly selected features.

(ii) Analysis

We can examine:

- Feature values.
- Classification accuracy.
- Misclassified samples.
- Decision scores.

(iii) Optimization

We can:

- Normalize the features.
- Learn better weights.
- Transform features.
- Remove irrelevant features.

Python Code:

```
1 # Q4: Discriminant Function Optimization
2
3 import numpy as np
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.metrics import accuracy_score
8
9 # -----
10 # 1. Create sample dataset
11 # -----
12 np.random.seed(42)
13
14 X = np.random.randn(200, 2)
15
16 # Create two classes
17 y = (
18     X[:, 0] + X[:, 1] > 0
19 ).astype(int)
20
21 X_train, X_test, y_train, y_test = train_test_split(
22     X, y,
23     test_size=0.25,
24     random_state=42
25 )
26
27 # -----
28 # 2. Poor manually selected weights
29 # -----
30 weights = np.array([0.1, 0.1])
31 bias = 0
32
33 scores = np.dot(X_test, weights) + bias
34
35 predictions = (scores >= 0).astype(int)
36
37 print("Accuracy Before Optimization:",
38       round(accuracy_score(y_test, predictions), 3))
39
40 # -----
41 # 3. Normalize features
42 # -----
43 scaler = StandardScaler()
44
45 X_train_scaled = scaler.fit_transform(X_train)
46 X_test_scaled = scaler.transform(X_test)
47
48 # -----
49 # 4. Learn better discriminant
50 # -----
51 model = LogisticRegression()
52 model.fit(X_train_scaled, y_train)
53
54 optimized_predictions = model.predict(X_test_scaled)
55
56 print("Accuracy After Optimization:",
57       round(
58         accuracy_score(
59             y_test,
60             optimized_predictions
61         ), 3
62       ))
63
64 print("\nOptimized Weights:")
65 print(model.coef_)
66
67 print("\nOptimized Bias:")
68 print(model.intercept_)
```

Output:

```
... Accuracy Before Optimization: 1.0
    Accuracy After Optimization: 1.0

    Optimized Weights:
    [[3.33473924 2.93718485]]

    Optimized Bias:
    [0.12076763]
```

Result: Learning suitable weights and normalizing features improves class separation.

Q5.A Bayesian logistic regression model developed for customer churn prediction is suffering from overfitting, resulting in poor generalization.

- (i) Identify signs and causes of overfitting in the model.
- (ii) Analyze model performance on training vs testing data.
- (iii) Apply debugging and regularization techniques (such as L1/L2 regularization or cross-validation) to improve model performance.

Answer)

Definitions:**Logistic Regression:**

Logistic Regression is a supervised classification algorithm used mainly for binary classification.

Overfitting:

Overfitting occurs when a model learns the training data too closely and performs poorly on unseen data.

Generalization:

Generalization is the ability of a model to perform well on new, unseen data.

Regularization:

Regularization adds a penalty to the model to reduce excessive complexity.

L1 Regularization:

L1 regularization adds the absolute values of coefficients to the loss function and can make some coefficients zero.

L2 Regularization:

L2 regularization penalizes the squared values of coefficients and reduces very large weights.

Cross-Validation:

Cross-validation divides the data into multiple parts and evaluates the model repeatedly.

(i) Signs of overfitting

Typical signs:

Training accuracy = very high

Testing accuracy = considerably lower

(ii) Analysis

We compare:

- Training accuracy.
- Testing accuracy.
- Cross-validation score.

(iii) Optimization

Use:

- L1 regularization.
- L2 regularization.
- Cross-validation.
- Feature selection.

Python Code:

```
1 # Q5: Logistic Regression Overfitting and Regularization
2
3 import numpy as np
4
5 from sklearn.datasets import make_classification
6 from sklearn.model_selection import (
7     train_test_split,
8     cross_val_score
9 )
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.metrics import accuracy_score
12
13 # -----
14 # 1. Create sample dataset
15 # -----
16 X, y = make_classification(
17     n_samples=500,
18     n_features=20,
19     n_informative=5,
20     n_redundant=10,
21     random_state=42
22 )
23
24 X_train, X_test, y_train, y_test = train_test_split(
25     X, y,
26     test_size=0.3,
27     random_state=42
28 )
29
30 # -----
31 # 2. Model with weak regularization
32 # -----
33 model_overfit = LogisticRegression(
34     C=1000,
35     max_iter=1000
36 )
37
38 model_overfit.fit(X_train, y_train)
39
40 train_pred = model_overfit.predict(X_train)
41 test_pred = model_overfit.predict(X_test)
42
43 print("Before Optimization")
44 print("Training Accuracy:",
45       round(accuracy_score(y_train, train_pred), 3))
46
47 print("Testing Accuracy:",
48       round(accuracy_score(y_test, test_pred), 3))
```

```

49
50 # -----
51 # 3. L2 Regularization
52 # -----
53 optimized_model = LogisticRegression(
54     C=0.1,
55     penalty="l2",
56     max_iter=1000
57 )
58
59 optimized_model.fit(X_train, y_train)
60
61 train_pred2 = optimized_model.predict(X_train)
62 test_pred2 = optimized_model.predict(X_test)
63
64 print("\nAfter L2 Regularization")
65 print("Training Accuracy:",
66       round(accuracy_score(y_train, train_pred2), 3))
67
68 print("Testing Accuracy:",
69       round(accuracy_score(y_test, test_pred2), 3))
70
71 # -----
72 # 4. Cross-validation
73 # -----
74 scores = cross_val_score(
75     optimized_model,
76     X,
77     y,
78     cv=5
79 )
80
81 print("\nCross Validation Scores:")
82 print(scores)
83
84 print("Average CV Accuracy:",
85       round(scores.mean(), 3))

```

Output:

```
... Before Optimization
    Training Accuracy: 0.837
    Testing Accuracy: 0.84

    After L2 Regularization
    Training Accuracy: 0.843
    Testing Accuracy: 0.847

    Cross Validation Scores:
    [0.82 0.85 0.88 0.83 0.83]
    Average CV Accuracy: 0.842
```

Result: Regularization reduces model complexity and helps the model generalize better.

Q6.A linear regression model built for predicting sales is performing poorly because the features (e.g., advertising budget in thousands and number of outlets in units) are on different scales.

- (i) Identify how feature scaling issues affect model performance.
- (ii) Analyze the impact of unscaled features on regression coefficients.
- (iii) Apply appropriate optimization techniques such as normalization or standardization to improve the model.

Answer)

Definitions:**Feature Scaling:**

Feature scaling transforms features so that their numerical ranges are comparable.

Standardization:

Standardization converts values using:

The resulting feature generally has mean 0 and standard deviation 1.

Normalization:

Normalization commonly converts values to a fixed range such as 0 to 1.

Regression Coefficient:

A regression coefficient represents the change in the predicted output for a change in a particular feature, while keeping other features constant.

(i) Problem

Suppose:

Advertising budget = 5000

Number of outlets = 20

The scales are very different.

(ii) Effect

Unscaled features can:

- Make optimization difficult.
- Cause coefficients to have very different magnitudes.
- Affect algorithms that depend on distances or gradient-based optimization.

(iii) Solution

Use standardization or normalization.

Python Code:

```
1 # Q6: Feature Scaling in Linear Regression
2
3 import pandas as pd
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LinearRegression
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import mean_squared_error
8
9 # -----
10 # 1. Sample input dataset
11 # -----
12 data = pd.DataFrame({
13     "Advertising_Budget": [
14         1000, 2000, 3000, 4000, 5000,
15         6000, 7000, 8000, 9000, 10000
16     ],
17     "Outlets": [
18         5, 8, 10, 12, 15,
19         18, 20, 22, 25, 30
20     ],
21     "Sales": [
22         120, 180, 240, 300, 350,
23         410, 470, 520, 590, 650
24     ]
25 })
26
27 print("Input Dataset:")
28 print(data)
29
30 X = data[["Advertising_Budget", "Outlets"]]
31 y = data["Sales"]
32
33 X_train, X_test, y_train, y_test = train_test_split(
34     X, y,
35     test_size=0.2,
36     random_state=42
37 )
38
39 # -----
40 # 2. Model without scaling
41 # -----
42 model1 = LinearRegression()
43 model1.fit(X_train, y_train)
44
45 pred1 = model1.predict(X_test)
46
47 print("\nWithout Scaling:")
48 print("Coefficients:", model1.coef_)
49 print("MSE:", round(mean_squared_error(y_test, pred1), 3))
50
51 # -----
52 # 3. Standardization
53 # -----
54 scaler = StandardScaler()
55
56 X_train_scaled = scaler.fit_transform(X_train)
57 X_test_scaled = scaler.transform(X_test)
58
59 model2 = LinearRegression()
60 model2.fit(X_train_scaled, y_train)
61
62 pred2 = model2.predict(X_test_scaled)
63
64 print("\nAfter Standardization:")
65 print("Coefficients:", model2.coef_)
66 print("MSE:", round(mean_squared_error(y_test, pred2), 3))
```

Output:

... Input Dataset:

	Advertising_Budget	Outlets	Sales
0	1000	5	120
1	2000	8	180
2	3000	10	240
3	4000	12	300
4	5000	15	350
5	6000	18	410
6	7000	20	470
7	8000	22	520
8	9000	25	590
9	10000	30	650

Without Scaling:

Coefficients: [0.0484375 3.59375]

MSE: 30.475

After Standardization:

Coefficients: [130.42196017 26.16289491]

MSE: 30.475

Result: Standardization puts features on comparable scales and makes coefficient comparison easier.

Q7.A binary classification model for fraud detection is biased towards the majority class, leading to poor detection of fraudulent cases.

(i) Identify the problem caused by class imbalance.

(ii) Evaluate model performance using suitable metrics (precision, recall, F1-score).

(iii) Apply optimization techniques such as resampling (oversampling/undersampling) or class weighting to improve performance.

Answer)

Definitions:

Class Imbalance:

Class imbalance occurs when one class has many more samples than another.

For fraud detection:

Normal transactions → majority

Fraudulent transactions → minority

Precision:

Precision tells us how many predicted fraud cases were actually fraud.

Recall:

Recall tells us how many actual fraud cases were detected.

F1-Score:**Class Weighting:**

Class weighting gives greater importance to the minority class during training.

(i) Problem

If 98% of transactions are normal and only 2% are fraudulent, a model may simply predict:

Every transaction = Normal

It can still obtain high accuracy but detect almost no fraud.

(ii) Evaluation

Use:

- Precision.
- Recall.
- F1-score.
- Confusion matrix.

(iii) Optimization

Use:

- Oversampling.

- Undersampling.
- Class weighting.

Here we use `class_weight="balanced"`.

Python Code:

```

1 # Q7: Fraud Detection with Class Imbalance
2
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import (
7     classification_report,
8     confusion_matrix
9 )
10
11 # -----
12 # 1. Create imbalanced dataset
13 # -----
14 X, y = make_classification(
15     n_samples=1000,
16     n_features=10,
17     n_informative=5,
18     n_redundant=2,
19     weights=[0.95, 0.05],
20     random_state=42
21 )
22
23 X_train, X_test, y_train, y_test = train_test_split(
24     X, y,
25     test_size=0.3,
26     random_state=42,
27     stratify=y
28 )
29
30 # -----
31 # 2. Model without balancing
32 # -----
33 model1 = LogisticRegression(max_iter=1000)
34
35 model1.fit(X_train, y_train)
36
37 pred1 = model1.predict(X_test)
38
39 print("Before Optimization")
40 print(confusion_matrix(y_test, pred1))
41 print(classification_report(y_test, pred1))
42
43 # -----
44 # 3. Class-weighted model
45 # -----
46 model2 = LogisticRegression(
47     class_weight="balanced",
48     max_iter=1000
49 )
50
51 model2.fit(X_train, y_train)
52
53 pred2 = model2.predict(X_test)
54
55 print("\nAfter Class Weighting")
56 print(confusion_matrix(y_test, pred2))
57 print(classification_report(y_test, pred2))

```


Output:

... Before Optimization

```
[[284  0]
 [ 15  1]]
```

	precision	recall	f1-score	support
0	0.95	1.00	0.97	284
1	1.00	0.06	0.12	16
accuracy			0.95	300
macro avg	0.97	0.53	0.55	300
weighted avg	0.95	0.95	0.93	300

After Class Weighting

```
[[209 75]
 [  2 14]]
```

	precision	recall	f1-score	support
0	0.99	0.74	0.84	284
1	0.16	0.88	0.27	16
accuracy			0.74	300
macro avg	0.57	0.81	0.56	300
weighted avg	0.95	0.74	0.81	300

Result: Class weighting makes the model pay more attention to fraudulent transactions and can improve fraud recall.

Q8.A Naïve Bayes classifier is underperforming because some input features are highly correlated, violating the independence assumption.

- Identify how correlated features affect Naïve Bayes performance.
- Analyze feature relationships in the dataset.
- Apply optimization techniques such as feature selection or dimensionality reduction to improve accuracy.

Answer)

Definitions:

Independence Assumption:

Naïve Bayes assumes that features are conditionally independent given the class.

For example:

$$P(A,B|C) = P(A|C) \times P(B|C)$$

Correlated Features:

Two features are correlated when changes in one feature are associated with changes in another.

Feature Selection:

Feature selection removes unnecessary or redundant features.

Dimensionality Reduction:

Dimensionality reduction reduces the number of input features while preserving important information.

PCA:

Principal Component Analysis (PCA) transforms correlated features into a smaller number of uncorrelated principal components.

(i) Problem

If:

Feature A $\leftrightarrow$ Feature B

are strongly correlated, Naïve Bayes may effectively count the same information more than once.

(ii) Analysis

We can calculate the correlation matrix.

(iii) Optimization

We can:

- Remove highly correlated features.
- Select useful features.

- Apply PCA.

Python Code:

```

1 # Q8: Naive Bayes with Correlated Features
2
3 import numpy as np
4 import pandas as pd
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.naive_bayes import GaussianNB
8 from sklearn.decomposition import PCA
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import accuracy_score
11
12 # -----
13 # 1. Create dataset
14 # -----
15 np.random.seed(42)
16
17 feature1 = np.random.normal(10, 2, 200)
18 feature2 = feature1 + np.random.normal(0, 0.2, 200)
19 feature3 = np.random.normal(5, 2, 200)
20
21 y = (feature1 + feature3 > 15).astype(int)
22
23 data = pd.DataFrame({
24     "Feature1": feature1,
25     "Feature2": feature2,
26     "Feature3": feature3,
27     "Target": y
28 })
29
30 print("Correlation Matrix:")
31 print(data.corr())
32
33 X = data.drop("Target", axis=1)
34 y = data["Target"]
35
36 X_train, X_test, y_train, y_test = train_test_split(
37     X, y,
38     test_size=0.25,
39     random_state=42
40 )
41
42 # -----
43 # 2. Naive Bayes with all features
44 # -----
45 model1 = GaussianNB()
46
47 model1.fit(X_train, y_train)
48

```

```

49 pred1 = model1.predict(X_test)
50
51 print("\nBefore Feature Optimization:")
52 print("Accuracy:",
53       round(accuracy_score(y_test, pred1), 3))
54
55 # -----
56 # 3. Remove correlated feature
57 # -----
58 X_reduced = data[
59     ["Feature1", "Feature3"]
60 ]
61
62 X_train2, X_test2, y_train2, y_test2 = train_test_split(
63     X_reduced, y,
64     test_size=0.25,
65     random_state=42
66 )
67
68 model2 = GaussianNB()
69
70 model2.fit(X_train2, y_train2)
71
72 pred2 = model2.predict(X_test2)
73
74 print("\nAfter Feature Selection:")
75 print("Accuracy:",
76       round(accuracy_score(y_test2, pred2), 3))
77
78 # -----
79 # 4. PCA alternative
80 # -----
81 scaler = StandardScaler()
82
83 X_scaled = scaler.fit_transform(X)
84
85 pca = PCA(n_components=2)
86
87 X_pca = pca.fit_transform(X_scaled)
88
89 print("\nPCA Explained Variance:")
90 print(pca.explained_variance_ratio_)

```

Output:

... Correlation Matrix:

	Feature1	Feature2	Feature3	Target
Feature1	1.000000	0.994586	-0.134197	0.547211
Feature2	0.994586	1.000000	-0.135543	0.540500
Feature3	-0.134197	-0.135543	1.000000	0.487517
Target	0.547211	0.540500	0.487517	1.000000

Before Feature Optimization:

Accuracy: 0.84

After Feature Selection:

Accuracy: 0.88

PCA Explained Variance:

[0.67663655 0.32155917]

Result: Removing redundant correlated features or applying PCA can make the feature representation more suitable for Naïve Bayes.

Q9.A logistic regression model is unable to clearly separate two classes due to overlapping feature distributions.

- (i) Identify the cause of overlapping decision boundaries.
- (ii) Analyze the feature space and classification errors.
- (iii) Improve the model using techniques such as feature transformation, polynomial features, or regularization.

Answer)

Definitions:

Logistic Regression:

Logistic Regression predicts the probability that an observation belongs to a particular class.

Feature Distribution:

A feature distribution shows how values of a feature are distributed across observations.

Overlapping Classes:

Overlapping classes occur when different classes have similar feature values.

Polynomial Features:

Polynomial features create additional features such as:

They allow a linear model to learn more complex relationships.

Regularization:

Regularization prevents the model from becoming unnecessarily complex.

(i) Cause

The classes may overlap because:

- Features are not sufficiently informative.
- The relationship is nonlinear.
- Important features are missing.
- A simple linear boundary is insufficient.

(ii) Analysis

We can compare:

- Classification accuracy.
- Confusion matrix.
- Decision boundary.
- Misclassified samples.

(iii) Optimization

Use polynomial features.

Python Code:

```
1 # Q9: Logistic Regression with Polynomial Features
2
3 from sklearn.datasets import make_moons
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.preprocessing import PolynomialFeatures
7 from sklearn.pipeline import Pipeline
8 from sklearn.metrics import accuracy_score
9
10 # -----
11 # 1. Create overlapping/nonlinear data
12 # -----
13 X, y = make_moons(
14     n_samples=300,
15     noise=0.25,
16     random_state=42
17 )
18
19 X_train, X_test, y_train, y_test = train_test_split(
20     X, y,
21     test_size=0.3,
22     random_state=42
23 )
24
25 # -----
26 # 2. Basic Logistic Regression
27 # -----
28 linear_model = LogisticRegression()
29
30 linear_model.fit(X_train, y_train)
31
32 pred1 = linear_model.predict(X_test)
33
34 print("Before Optimization:")
35 print("Accuracy:",
36       round(accuracy_score(y_test, pred1), 3))
37
38 # -----
39 # 3. Polynomial Logistic Regression
40 # -----
41 poly_model = Pipeline([
42     ("polynomial",
43      PolynomialFeatures(degree=3)),
44     ("logistic",
45      LogisticRegression(max_iter=1000))
46 ])
47
48 poly_model.fit(X_train, y_train)
49
50 pred2 = poly_model.predict(X_test)
51
52 print("\nAfter Polynomial Feature Transformation:")
53 print("Accuracy:",
54       round(accuracy_score(y_test, pred2), 3))
```

Output:

```
... Before Optimization:  
Accuracy: 0.878
```

```
After Polynomial Feature Transformation:  
Accuracy: 0.889
```

Result: Polynomial features allow Logistic Regression to model nonlinear class boundaries more effectively.

Q10. A discriminant analysis model is too simple and fails to capture the complexity of the dataset, resulting in low accuracy.

- (i) Identify the signs of underfitting in the model.
- (ii) Analyze model bias and feature limitations.
- (iii) Apply optimization strategies such as adding features, increasing model complexity, or using nonlinear transformations.

Answer)

Definitions:**Discriminant Analysis:**

Discriminant Analysis is a classification technique that uses statistical properties of classes to separate observations.

Underfitting:

Underfitting occurs when a model is too simple to capture important patterns in the data.

Bias:

Bias is the error caused by overly simple assumptions in a model.

Model Complexity:

Model complexity represents the ability of a model to represent complex relationships.

Feature Engineering:

Feature engineering means creating or transforming features to provide more useful information to the model.

(i) Signs of Underfitting

Common signs are:

Training accuracy → Low

Testing accuracy → Low

Unlike overfitting, both training and testing performance may be poor.

(ii) Causes

- Too few features.
- Oversimplified model.
- Important nonlinear relationships ignored.
- Insufficient feature engineering.

(iii) Optimization

We can:

- Add relevant features.
- Transform existing features.
- Increase model complexity.
- Use nonlinear features.

Python Code:

```

1 # Q10: Discriminant Analysis and Underfitting
2
3 import numpy as np
4
5 from sklearn.datasets import make_classification
6 from sklearn.model_selection import train_test_split
7 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
8 from sklearn.preprocessing import PolynomialFeatures
9 from sklearn.pipeline import Pipeline
10 from sklearn.metrics import accuracy_score
11
12 # -----
13 # 1. Create sample dataset
14 # -----
15 X, y = make_classification(
16     n_samples=500,
17     n_features=6,
18     n_informative=5,
19     n_redundant=0,
20     class_sep=0.7,
21     random_state=42
22 )
23
24 X_train, X_test, y_train, y_test = train_test_split(
25     X, y,
26     test_size=0.3,
27     random_state=42
28 )
29
30 # -----
31 # 2. Basic LDA model
32 # -----
33 lda_model = LinearDiscriminantAnalysis()
34
35 lda_model.fit(
36     X_train[:, :2],
37     y_train
38 )
39
40 train_pred = lda_model.predict(
41     X_train[:, :2]
42 )
43
44 test_pred = lda_model.predict(
45     X_test[:, :2]
46 )
47

```

```

48 print("Before Optimization:")
49 print("Training Accuracy:",
50       round(
51           accuracy_score(y_train, train_pred), 3
52       ))
53
54 print("Testing Accuracy:",
55       round(
56           accuracy_score(y_test, test_pred), 3
57       ))
58
59 # -----
60 # 3. Use more features
61 # -----
62 optimized_model = LinearDiscriminantAnalysis()
63
64 optimized_model.fit(
65     X_train,
66     y_train
67 )
68
69 train_pred2 = optimized_model.predict(X_train)
70 test_pred2 = optimized_model.predict(X_test)
71
72 print("\nAfter Adding Relevant Features:")
73 print("Training Accuracy:",
74       round(
75           accuracy_score(y_train, train_pred2), 3
76       ))
77
78 print("Testing Accuracy:",
79       round(
80           accuracy_score(y_test, test_pred2), 3
81       ))
82
83 # -----
84 # 4. Show model coefficients
85 # -----
86 print("\nLDA Coefficients:")
87 print(optimized_model.coef_)

```

Output:

```
... Before Optimization:
    Training Accuracy: 0.763
    Testing Accuracy: 0.753

    After Adding Relevant Features:
    Training Accuracy: 0.783
    Testing Accuracy: 0.747

    LDA Coefficients:
    [[-0.98331792  1.02059512  0.12296215 -0.73650531 -0.06920764 -0.32590199]]
```

Result: Adding informative features can reduce underfitting and improve classification performance.